第 11 讲：双指针——用两个位置共同描述搜索范围

0. 本讲的核心问题

很多数组和字符串题的暴力方法会枚举两个位置：

```
[python]
for i in range(n):
    for j in range(i + 1, n):
        ...
```

这通常需要 $O(n^2)$ 时间。

双指针的核心是：

> 利用数据的顺序、单调性或处理进度，让两个位置有目的地移动，从而排除大量不可能的组合。

双指针不是一种数据结构，而是一类变量设计方法。常见形态包括：

1. 左右指针：从两端向中间移动；
2. 快慢指针：同向移动，速度或职责不同；
3. 读写指针：一个读取输入，一个维护输出位置；
4. 两序列指针：分别指向两个数组当前未处理位置。

1. 双指针能够成立的前提

关键不是“用了两个变量”，而是移动一个指针后，能安全排除一批答案。

例如有序数组：

```
[1, 3, 4, 7, 9]
```

左右两端和太小：

```
nums[left] + nums[right] < target
```

如果继续保留较小的 `nums[left]`，把右指针左移只会让和更小，不可能得到目标。因此可以安全地执行：

```
[python]
left += 1
```

这就是单调性提供的排除依据。

每次移动前都应该能回答：

> 为什么被跳过的那些情况不可能是答案？

2. 左右指针：验证回文串

一个字符串是回文，当且仅当两端字符相同，并且内部子串也是回文。

```
[python]
def is_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        if text[left] != text[right]:
            return False

        left += 1
        right -= 1

    return True
```

2.1 循环不变量

每轮开始时：

> `text[:left]` 与 `text[right+1:]` 已经完成配对并且全部匹配；只剩 `[left, right]` 尚未确认。

2.2 为什么条件是 `left < right`

- 奇数长度时，最终剩一个中间字符，不需要与自己比较；
- 偶数长度时，指针会相邻后交错；
- 当 `left >= right`，所有需要比较的字符对均已通过。

时间 `O(n)`，额外空间 `O(1)`。

---

3. 忽略非字母数字的回文串

题目可能要求忽略标点和大小写：

```
[python]
def is_valid_palindrome(text):
    left = 0
    right = len(text) - 1

    while left < right:
        while left < right and not text[left].isalnum():
            left += 1

        while left < right and not text[right].isalnum():
            right -= 1

        if text[left].lower() != text[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

这里有三层循环，但总时间仍是 `O(n)`：每个指针只会单向移动，任何位置最多被各自经过一次。

---

4. 有序数组的两数之和

```
[python]
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1

    return []
```

4.1 为什么这样移动

数组已经升序排列。

- 和太小：只有增大较小数才可能达到目标，因此左指针右移；
- 和太大：只有减小较大数才可能达到目标，因此右指针左移。

4.2 与哈希表两数之和比较

方法	前提	时间	空间
哈希表	无需有序	`O(n)` 平均	`O(n)`
左右指针	数组有序	`O(n)`	`O(1)`

如果为了使用双指针先排序，要考虑排序 `O(n log n)`，并且原下标会变化。

---

5. 盛最多水的容器

两条竖线下标为 `left`、`right`，面积为：

```
   ×
(right - left) × min(height[left], height[right])
```

```
[python]
def max_area(height):
    left = 0
    right = len(height) - 1
    best = 0

    while left < right:
        width = right - left
        current_height = min(height[left], height[right])
        best = max(best, width * current_height)

        if height[left] <= height[right]:
            left += 1
        else:
            right -= 1

    return best
```

5.1 为什么移动较短的一侧

假设左边更短。保留左边并移动右边：

- 宽度一定变小；
- 高度仍然不可能超过左边这条短线的限制。

因此面积不可能变大，可以排除所有“左边不变、右边向内”的组合。

只有移动短边，才有机会遇到更高的边，弥补宽度下降。

这道题的关键不是公式，而是这个排除证明。

---

6. 读写指针：原地移动零

要求把所有零移动到末尾，同时保持非零元素相对顺序。

可以把 `write` 理解为：

> 下一个非零元素应该写入的位置。

```
[python]
def move_zeroes(nums):
    write = 0

    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write] = nums[read]
            write += 1

    while write < len(nums):
        nums[write] = 0
        write += 1
```

6.1 循环不变量

处理 `read` 之前：

```
[python]
nums[:write]
```

扫描结束后，前缀已经正确，只需把剩余位置填零。

6.2 交换写法

```
[python]
def move_zeroes(nums):
    slow = 0

    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

这种写法一步完成移动和填零。两种都正确，但必须理解 `slow` 的含义。

---

7. 删除有序数组中的重复项

有序数组中的相同元素相邻。保留每个值的第一个：

```
[python]
def remove_duplicates(nums):
```

```
    if not nums:
        return 0

    write = 1

    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1

    return write
```

返回 `write` 后，有效结果是：

```
[python]
nums[:write]
```

循环不变量：

> `nums[:write]` 是已处理前缀去重后的结果；`write` 是下一个新值的写入位置。

不要要求数组后半部分保持原值。原地题通常只规定前 `k` 个位置有效。

---

8. 两个有序数组的交集式扫描

两个指针分别指向两个有序序列：

```
[python]
def intersection_sorted(nums1, nums2):
    i = 0
    j = 0
    result = []

    while i < len(nums1) and j < len(nums2):
        if nums1[i] == nums2[j]:
            result.append(nums1[i])
            i += 1
            j += 1
        elif nums1[i] < nums2[j]:
            i += 1
        else:
            j += 1

    return result
```

较小元素不可能与对方当前或之后的更大元素相等，因此可以安全跳过。

这类模式也用于：

- 合并两个有序数组；
- 比较两个字符串处理结果；
- 求区间交集；
- 归并排序中的合并阶段。

---

9. 三数之和

题目：找出不重复的三元组，使和为 0。

暴力三层循环为 `O(n³)`。排序后，可以固定第一个数，再对剩余部分使用左右指针：

```
[python]
def three_sum(nums):
    nums.sort()
    result = []

    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        if nums[i] > 0:
            break

        left = i + 1
        right = len(nums) - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]

            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1

                while left < right and nums[left] == nums[left - 1]:
                    left += 1

                while left < right and nums[right] == nums[right + 1]:
                    right -= 1

    return result
```

9.1 去重分成两层

固定值去重：

```
if i > 0 and nums[i] == nums[i - 1]:
    continue
```

找到答案后，左右值去重：

```
[python]
while left < right and nums[left] == nums[left - 1]:
    left += 1
```

为什么不能一看到 `nums[left] == nums[left-1]` 就无条件跳过？

因为在尚未找到答案时，当前重复值可能仍需与不同的右值组合。去重必须与枚举逻辑对应。

9.2 复杂度

```
O(n log n)
i         O(n)   O(n)
 O(n²)
                       O(1)   O(log n)
```

---

10. 快慢指针的其他含义

第 6 讲链表已经见过：

- 快慢指针找中点；
- Floyd 算法检测环。

这里要统一认识：快慢指针不一定真的是“速度不同”。它们也可以职责不同：

```
fast / read
slow / write
```

做题时先描述职责，不要只写“定义两个指针”。

---

11. 双指针与滑动窗口的区别

它们都可能使用 `left` 和 `right`，但目的不同。

双指针

两个位置协作排除组合或原地整理数据：

滑动窗口

`left` 和 `right` 共同定义一个连续区间，并维护区间内部的统计信息：

k

滑动窗口是同向双指针的一个特殊模式，但它强调“窗口状态”。下一讲会专门学习。

---

12. 常见错误

12.1 没有利用有序性却强行双指针

无序两数之和不能根据和的大小决定移动方向。应使用哈希表，或先排序并处理原下标问题。

12.2 移错了指针

不能凭口诀移动。必须根据单调性解释为什么某一侧可以排除。

12.3 `left <= right` 与 `left < right` 混淆

两数配对不能使用同一元素两次时通常是 `left < right`。处理单个候选位置时可能需要 `<=`。

12.4 原地写入覆盖尚未读取的数据

读写指针中通常保证 `write <= read`，这样写入不会破坏未来数据。需要反向合并时，则从数组末尾向前写。

12.5 忘记结果只在前缀有效

删除重复项返回长度 `k`，答案是 `nums[:k]`，不是整个数组。

12.6 三数之和去重位置错误

固定数去重与左右指针去重是不同层次，不能混为一条判断。

12.7 指针没有保证前进

每条分支都要让至少一个指针发生变化，否则可能死循环。

---

13. 本讲检查表

```
1.
2.
3.
4.
5.
6.        left < right    left <= right
7.
8.
9.
10.
```

14. 本讲总结

1. 双指针利用顺序或单调性减少重复枚举。
2. 左右指针常用于有序数组、回文和区间收缩。
3. 读写指针常用于稳定地原地整理数组。
4. 两序列指针每次排除当前较小者，常用于合并和比较。
5. 三数之和是“排序 + 固定一个数 + 两数双指针”。
6. 不要背移动口诀；每次移动都要能够说明排除了哪些不可能情况。